

Route 53 and Application Load Balancer

DNS · ALIAS Records · ALB Routing · HTTPS · WAF · Health Checks · ShopFlow

AWS Tutorial · ShopFlow

13-01

How Traffic Reaches ShopFlow — DNS and Load Balancing

■ Route 53 and Application Load Balancer

When a customer types shopflow.com in their browser, two AWS services direct that traffic: Route 53 (DNS) translates the domain name to an IP address, and ALB (Application Load Balancer) receives the HTTPS request and routes it to the correct ECS task. Together they enable high availability, HTTPS, and traffic splitting.

Example: ShopFlow traffic path: Browser -> DNS lookup (Route 53: shopflow.com -> ALB IP) -> HTTPS to ALB -> ALB listener rule matches /products/* -> routes to product-svc ECS target group -> one of 6 running product-svc tasks handles the request.

Service	Layer	Responsibility	ShopFlow Config
Route 53	DNS (Layer 7 — application)	Domain name -> IP address resolution; health-checks for failover	ShopFlow domain (for ShopFlow)
ALB (Application LB)	HTTP/HTTPS (Layer 7)	HTTPS termination; path-based routing (/products/*)	ALB listener rules based on path
NLB (Network LB)	TCP/UDP (Layer 4)	Ultra-low latency; static IP addresses; TLS pass-through	ShopFlow not used (if needed for all use cases)
CloudFront	CDN (Layer 7 — edge)	Static asset caching at edge; WAF integration; TLS offload	ShopFlow before ALB (CloudFront -> ALB)

13-02

Route 53 — DNS Management Basics

Route 53 is AWS's managed DNS service. Understanding DNS record types is essential before configuring your domain for ShopFlow.

Record Type	Purpose	ShopFlow Example
A record	Maps hostname to IPv4 address	api.shopflow.com -> 52.1.2.3 (rarely used directly; use ALIAS instead)
AAAA record	Maps hostname to IPv6 address	api.shopflow.com -> 2001:db8::1 (IPv6)
CNAME record	Maps hostname to another hostname; cannot be shopflow.com (redirect www to root)	www.shopflow.com -> shopflow.com (redirect www to root)
ALIAS record	AWS-specific; maps hostname to AWS resource	shopflow.com -> shopflow-123-us-east-1.amazonaws.com (ALIAS; not CNAME)
NS record	Name server records; delegates DNS authority	shopflow.com to ns-123-awsdns-00.com (auto-created when you create zone)
MX record	Mail exchange; routes email for domain	shopflow.com MX 10 mail.shopflow.com (email routing; separate from www)
TXT record	Arbitrary text; used for domain verification and SPF/DKIM	shopflow.com TXT "v=spf1 include:amazonses.com ~all" (email authentication)

13-03

Route 53 — Create Hosted Zone and DNS Records

A Hosted Zone is Route 53's container for DNS records for your domain. You create one per domain and add all records to it.

■ Create Hosted Zone and Point to ALB

1. Route 53 Console -> Hosted zones -> Create hosted zone
2. Domain name: shopflow.com
3. Type: Public hosted zone
4. Click "Create hosted zone"
5. Note the 4 NS (name server) records created automatically
6. IMPORTANT: Update your domain registrar (GoDaddy, Namecheap, etc.) to use these NS records
7. This delegates DNS control from your registrar to Route 53
8. Now create the ALIAS record: Create record
9. Record name: (leave blank — this is the root shopflow.com)
10. Record type: A
11. Toggle "Alias" ON
12. Route traffic to: Alias to Application and Classic Load Balancer
13. Region: us-east-1
14. Load balancer: select shopflow-alb
15. Click "Create records"
16. Test: dig shopflow.com (should return ALB IP addresses)

■ aws > Route 53 > Hosted zones > shopflow.com > Records

DNS Records

■ shopflow.com — Public hosted zone

- NS shopflow.com ns-123.awsdns-00.com, ns-456.awsdns-11.net, ns-789.awsdns-22.org, ns-012.awsdns-33.co.uk
 - SOA shopflow.com ns-123.awsdns-00.com
 - A shopflow.com -> ALIAS shopflow-alb-123.us-east-1.elb.amazonaws.com
 - A www.shopflow.com -> ALIAS shopflow-alb-123.us-east-1.elb.amazonaws.com
 - A api.shopflow.com -> ALIAS shopflow-alb-123.us-east-1.elb.amazonaws.com
 - CNAME admin.shopflow.com -> shopflow-admin.netlify.app
- 4 records | Zone created | NS delegated to Route 53

13-04

HTTPS with ACM — Free SSL/TLS Certificates

AWS Certificate Manager (ACM) provides free SSL/TLS certificates for HTTPS. Certificates automatically renew. ShopFlow uses a wildcard certificate (*.shopflow.com) covering all subdomains.

■ Request and Attach ACM Certificate

1. ACM Console -> Request certificate
2. Certificate type: Public
3. Domain names: shopflow.com (add both root and wildcard)
4. Click "Add another name": *.shopflow.com
5. Validation method: DNS validation (recommended — auto-renews)
6. Click "Request"
7. On the pending certificate page: click "Create records in Route 53"
8. Route 53 auto-creates the CNAME validation records
9. Wait 5-10 minutes for certificate to show "Issued" status
10. Go to ALB Console -> shopflow-alb -> Listeners tab
11. Select HTTPS:443 listener -> Edit -> Default SSL certificate
12. Choose ACM certificate: shopflow.com + *.shopflow.com
13. Save -> ALB now terminates HTTPS with your certificate

INFO Wildcard Certificate

A wildcard certificate (*.shopflow.com) covers all subdomains: api.shopflow.com, admin.shopflow.com, staging.shopflow.com — with a single certificate. ACM issues and renews wildcard certificates for free. The certificate is attached to the ALB, which terminates TLS. Traffic inside your VPC (ALB to ECS tasks) can use HTTP (port 80) because it is on the private network.

13-05

ALB Listener Rules — Path-Based Routing

ALB listener rules route HTTPS requests to different target groups based on URL path or hostname. ShopFlow uses one ALB with path-based rules to route traffic to 4 different microservices — no need for a separate ALB per service.

■ aws > EC2 > Load Balancers > shopflow-alb > Listeners > HTTPS:443 > Rules

Listener Rules

■ HTTPS:443 Listener — Rules (evaluated in priority order)

- Priority 1: IF Path is /products/* THEN Forward to product-svc-tg
 - Priority 2: IF Path is /orders/* THEN Forward to order-svc-tg
 - Priority 3: IF Path is /cart/* THEN Forward to cart-svc-tg
 - Priority 4: IF Path is /payments/* THEN Forward to payment-svc-tg
 - Priority 5: IF Path is /health THEN Return fixed 200 OK (ALB health check)
- Default action: Return 404 (no rule matched)
- SSL Policy: ELBSecurityPolicy-TLS13-1-2-2021-06 (TLS 1.3 preferred)
- All rules active | Certificate: *.shopflow.com (ACM)

```
// CDK: ALB + HTTPS listener with path-based routing rules
ApplicationLoadBalancer alb =
ApplicationLoadBalancer.Builder.create(this, "ShopFlowALB") .loadBalancerName("shopflow-alb")
```

```
.vpc(vpc) .internetFacing(true) .securityGroups(List.of(albSg)) .build(); ApplicationListener
httpsListener = alb.addListener("HTTPS", BaseApplicationListenerProps.builder() .port(443)
.protocol(ApplicationProtocol.HTTPS)
.certificates(List.of(ListenerCertificate.fromArn(certArn)))
.sslPolicy(SslPolicy.RECOMMENDED_TLS) .defaultAction(ListenerAction.fixedResponse(404,
FixedResponseOptions.builder().contentType("application/json") .messageBody("{\"error\":\"Not
found\"}").build())) .build()); // Redirect HTTP to HTTPS alb.addListener("HTTP",
BaseApplicationListenerProps.builder() .port(80)
.defaultAction(ListenerAction.redirect(RedirectOptions.builder()
.protocol("HTTPS").port("443").permanent(true).build())) .build()); // Path-based rules
httpsListener.addAction("ProductsRule", AddApplicationActionProps.builder() .priority(1)
.conditions(List.of(ListenerCondition.pathPatterns(List.of("/products/*"))))
.action(ListenerAction.forward(List.of(productSvcTargetGroup))) .build());
```

13-06

Target Groups and Health Checks

A Target Group is the set of targets (ECS tasks) that receive traffic from an ALB rule. Health checks determine which targets are healthy and eligible to receive traffic. Unhealthy targets are automatically removed until they recover.

```
// CDK: Target Group with health check ApplicationTargetGroup productSvcTg =
ApplicationTargetGroup.Builder .create(this, "ProductSvcTG")
.targetGroupName("shopflow-product-svc") .vpc(vpc) .port(8080)
.protocol(ApplicationProtocol.HTTP) .targetType(TargetType.IP) // Fargate: IP-based, not
instance-based .healthCheck(HealthCheckProps.builder() .path("/actuator/health") // Spring Boot
health endpoint .protocol(Protocol.HTTP) .port("8080") .healthyHttpCodes("200")
.interval(Duration.seconds(30)) // check every 30 seconds .timeout(Duration.seconds(5)) // must
respond within 5s .healthyThresholdCount(2) // 2 consecutive successes -> healthy
.unhealthyThresholdCount(3) // 3 consecutive failures -> unhealthy .build())
.deregistrationDelay(Duration.seconds(30)) // drain for 30s before removing
.stickinessCookieDuration(Duration.minutes(0)) // no sticky sessions (stateless) .build();
```

Health Check Setting	Value	Reasoning
Path	/actuator/health	Spring Boot Actuator: checks DB, Redis, disk — only 200 when fully ready
Interval	30 seconds	Balance: detect failures within ~90s (3 checks) without overloading tasks with h
Timeout	5 seconds	If Spring Boot does not respond in 5s, something is wrong; count as failure
Unhealthy threshold	3 failures	Avoid removing task on transient network blip; 3 consecutive = truly unhealthy
Deregistration delay	30 seconds	ALB stops sending new requests; waits 30s for in-flight requests to complete

13-07

Route 53 Health Checks — Automatic DNS Failover

Route 53 health checks monitor your endpoints and automatically update DNS if an endpoint becomes unhealthy. ShopFlow uses failover routing to switch to a DR region if the primary us-east-1 ALB becomes unreachable.

■ Configure Route 53 Failover Routing

1. Route 53 -> Health checks -> Create health check
2. Name: shopflow-us-east-1-primary
3. Monitor: Endpoint
4. Protocol: HTTPS | Domain: shopflow-alb-123.us-east-1.elb.amazonaws.com
5. Path: /health | Port: 443
6. Advanced: Failure threshold = 3, Request interval = 30 seconds
7. Create health check
8. Go to Hosted zones -> shopflow.com -> Edit the A record for shopflow.com
9. Routing policy: Failover
10. Failover record type: Primary
11. Associate with health check: shopflow-us-east-1-primary
12. Create a second record: shopflow.com -> ALIAS to us-west-2 ALB
13. Routing policy: Failover | Failover type: Secondary
14. If primary health check fails, Route 53 returns the secondary ALB IP

13-08

Route 53 Routing Policies — Beyond Simple DNS

Route 53 supports 7 routing policies beyond simple (one record, one IP). ShopFlow uses several for different purposes.

Routing Policy	How It Works	ShopFlow Use Case
Simple	One record returns one value (or multiple for random selection)	Advised selection for production scale; used in early development
Weighted	Multiple records with weights; traffic split proportionally	Blue/green deployment: weight=90 primary, weight=10 new version
Latency	Route to the region with lowest network latency	ShopFlow user -> us-east-1 for East Coast users, eu-west-1 for European users
Failover	Primary active; secondary on standby; health-checked switchover	ShopFlow us-east-1 primary; us-west-2 DR if primary fails
Geolocation	Route based on user country or continent	Block traffic from sanctioned countries; route CN users to China region
Geoproximity	Route based on geographic proximity to resource	Shift 20% adjust traffic to us-east-1 during a capacity event in eu-west-1
Multi-value answer	Return up to 8 IPs, filtering out unhealthy endpoints	Not used by ShopFlow — ALB provides this functionality

13-09

Route 53 + ALB with CDK — Complete Setup

CDK manages the full DNS + ALB stack: hosted zone, ACM certificate, ALB, listener rules, and Route 53 records. The `CertificateManager.fromDns()` construct automatically creates the Route 53 validation records.

```
// CDK: Complete Route 53 + ACM + ALB stack // 1. Look up existing hosted zone (already
registered) HostedZone hostedZone = HostedZone.fromLookup(this, "ShopFlowZone",
HostedZoneProviderProps.builder().domainName("shopflow.com").build()); // 2. ACM certificate
with automatic DNS validation Certificate certificate = Certificate.Builder.create(this,
"ShopFlowCert").domainName("shopflow.com").subjectAlternativeNames(List.of("*.shopflow.com"))
.validation(CertificateValidation.fromDns(hostedZone)).build(); // 3. ALB (already created in
ch11) ApplicationLoadBalancer alb = ...; // from ECS chapter // 4. Route 53 ALIAS record
pointing to ALB ARecord.Builder.create(this, "ShopFlowAliasRecord").zone(hostedZone)
.recordName("shopflow.com").target(RecordTarget.fromAlias(new LoadBalancerTarget(alb)))
```

```
.build(); // Also create www subdomain ARecord.Builder.create(this, "wwwAliasRecord")
.zone(hostedZone) .recordName("www") .target(RecordTarget.fromAlias(new
LoadBalancerTarget(alb))) .build(); // 5. Route 53 Health Check + Failover (advanced – done in
console or CloudFormation) CfnHealthCheck primaryHealthCheck =
CfnHealthCheck.Builder.create(this, "PrimaryHC")
.healthCheckConfig(CfnHealthCheck.HealthCheckConfigProperty.builder() .type("HTTPS")
.fullyQualifiedDomainName(alb.getLoadBalancerDnsName()) .resourcePath("/health") .port(443)
.failureThreshold(3) .requestInterval(30) .build()) .build();
```

13-10

ALB Access Logs — Traffic Analysis with Athena

ALB access logs record every request: source IP, URL, response code, latency, target IP. Stored in S3 and queryable with Athena, they are essential for debugging 5xx errors, investigating security events, and capacity planning.

■ Enable ALB Access Logs

1. EC2 Console -> Load Balancers -> shopflow-alb
2. Attributes tab -> Edit
3. Access logs: Enable
4. S3 location: s3://shopflow-alb-logs/alb/ (bucket must pre-exist)
5. Save changes
6. --- Query with Athena (run after logs accumulate) ---
7. Athena Console -> Create table from S3 -> use ALB log format
8. Or run: CREATE EXTERNAL TABLE alb_logs (...) PARTITIONED BY (year, month, day)
9. Query: SELECT request_url, COUNT(*) as count FROM alb_logs WHERE elb_status_code=503 GROUP BY request_url ORDER BY count DESC LIMIT 20
10. Find which endpoints are generating 503 errors most frequently

```
-- Athena: Find slowest ShopFlow API endpoints (P99 > 1 second) SELECT
regexp_extract(request_url, '(https://[^\s/]+)/([^\s/]+)', 2) as path, COUNT(*) as request_count,
ROUND(approx_percentile(target_processing_time, 0.99) * 1000, 0) as p99_ms,
ROUND(AVG(target_processing_time) * 1000, 0) as avg_ms, SUM(CASE WHEN elb_status_code >= 500
THEN 1 ELSE 0 END) as errors FROM alb_logs WHERE year='2024' AND month='11' AND day='29' GROUP
BY 1 HAVING approx_percentile(target_processing_time, 0.99) > 1.0 ORDER BY p99_ms DESC LIMIT
20;
```

13-11

AWS WAF on ALB — Layer 7 Protection

AWS WAF (Web Application Firewall) attaches to your ALB and blocks malicious requests before they reach your ECS tasks. ShopFlow uses WAF to block SQL injection, XSS, and rate-limit abusive IPs.

```
// CDK: WAF WebACL attached to ALB CfnWebACL webAcl = CfnWebACL.Builder.create(this,
"ShopFlowWAF") .name("shopflow-waf") .scope("REGIONAL") // for ALB (CLOUDFRONT for CloudFront
distributions) .defaultAction(CfnWebACL.DefaultActionProperty.builder()
.allow(CfnWebACL.AllowActionProperty.builder().build()).build()) .rules(List.of( // Rule 1:
AWS Managed Rules – Common Rule Set (OWASP top 10) CfnWebACL.RuleProperty.builder()
.name("AWSManagedRulesCommonRuleSet") .priority(1)
.statement(CfnWebACL.StatementProperty.builder() .managedRuleGroupStatement(
CfnWebACL.ManagedRuleGroupStatementProperty.builder()
.vendorName("AWS").name("AWSManagedRulesCommonRuleSet") .build()) .build())
.overrideAction(CfnWebACL.OverrideActionProperty.builder() .none(Map.of()).build()) // use
managed rule's block action .visibilityConfig(...) .build(), // Rule 2: Rate limit per IP (1000
req/5min) CfnWebACL.RuleProperty.builder() .name("RateLimitPerIP") .priority(2)
.action(CfnWebACL.RuleActionProperty.builder()
.block(CfnWebACL.BlockActionProperty.builder().build()).build())
.statement(CfnWebACL.StatementProperty.builder()
```

```

.rateBasedStatement(CfnWebACL.RateBasedStatementProperty.builder() .aggregateKeyType("IP")
.limit(1000) // block IP if > 1000 requests in 5 minutes .build()) .build())
.visibilityConfig(...) .build())) .build(); // Associate WAF with ALB
CfnWebACLAssociation.Builder.create(this, "WafALBAssoc") .resourceArn(alb.getLoadBalancerArn())
.webAclArn(webAcl.getAttrArn()) .build();

```

13-12

Weighted Routing — Blue/Green Deployments with Route 53

Route 53 weighted routing lets you send a percentage of traffic to a new version (green) while keeping most traffic on the existing version (blue). This enables canary deployments: start at 5%, verify metrics, then gradually shift to 100%.

■ Blue/Green Canary Deploy with Route 53 Weights

1. Deploy new version (green) to a second ALB or ECS service
2. Route 53 -> Hosted zones -> shopflow.com
3. Edit the shopflow.com A record: currently weight=100 pointing to primary ALB
4. Add a new A record: shopflow.com -> ALIAS to green ALB, weight=0
5. Deploy: change green weight to 5, primary weight to 95
6. Monitor: 5% of users hit green; check error rates, P99 latency in CloudWatch
7. If green metrics are healthy: 10/90 -> 25/75 -> 50/50 -> 0/100
8. If green has errors: set green weight=0 (instant rollback)
9. Delete old blue ALB once green is fully rolled out
10. Total traffic shift time: however long you monitor each percentage step

Stage	Blue Weight	Green Weight	When to Advance
Initial canary	95	5	Wait 30 min; verify error rate < 0.1%, P99 similar to blue
Extended canary	75	25	Wait 1 hour; verify metrics stable across traffic pattern
Majority green	25	75	Wait 30 min; monitor for edge cases in less-common flows
Full cutover	0	100	Green is production; keep blue 24 hours for instant rollback
Cleanup	Deleted	100	Delete blue ALB and old ECS service version

13-13

Sticky Sessions and Connection Draining

Sticky sessions (session affinity) send a user's requests to the same target for the duration of a session. ShopFlow does NOT use sticky sessions — all state is in ElastiCache Redis, making ECS tasks fully stateless. Understanding why is important for beginners who default to enabling stickiness.

Approach	Sticky Sessions (DO NOT use for ShopFlow)	Stateless with Redis (ShopFlow approach)
User session storage	In-memory on the ECS task that served the first request	Redis reads from Redis on every request; any task can serve
What happens if the task crashes	User loses session — forced re-login	Another task reads session from Redis; user never notices the crash
Scaling behavior	ALB must route user to specific task — limits load distribution	Any new task can serve any user immediately — perfect load distribution
Deployment	Draining sticky task means all its users lose session	Rolling deploy: users seamlessly continue on new tasks via Redis
Cost	No extra cost — but wastes capacity (some tasks idle, some overloaded)	Redis is a one-time cost, then a small, steady operational improvement

WARN When Sticky Sessions ARE Appropriate

Sticky sessions are appropriate for legacy applications that store state in-process and cannot be refactored. They are also useful for WebSocket connections — ALB must route all WebSocket messages for a connection to the same target. ShopFlow WebSocket connections (real-time order status) use NLB instead of ALB for persistent TCP connections, avoiding the sticky session problem entirely.

13-14

ShopFlow Traffic Architecture — End to End

The complete traffic flow for a ShopFlow product page request, from the customer browser to the ECS task and back.

ShopFlow Traffic Architecture (Full Request Flow)



Step	Latency Added	Cumulative
DNS resolution (Route 53)	< 1ms (cached by OS/ISP for TTL duration)	
TCP + TLS to CloudFront edge	10-50ms (depends on user location)	~30ms
CloudFront -> ALB (AWS backbone)	< 5ms (internal AWS network)	~35ms
WAF inspection	< 1ms	~36ms
ALB routing	< 1ms	~37ms
ECS product-svc: Redis cache hit	5-10ms	~47ms
Response back to browser	10-50ms (reverse network path)	~80ms total

Question	Answer
Should I use CNAME or ALIAS for my ALB?	ALIAS (An Alias record with alias enabled) for root domain (shopflow.com) — CNAME is not allowed at the root of a domain.
My ALB is returning 502 Bad Gateway. What's wrong?	502: Health check received a response from the target that it could not process, OR could not connect to target.
How do I test ALB routing rules before deploying?	Test feature in the Console: Listeners -> Rules -> Actions -> Test. Enter a test host and path.
Best Practice	Anti-Pattern to Avoid
Use ALIAS records to point root domain to ALB — no CNAMEs.	Using CNAMEs for root domain — technically invalid; many DNS resolvers reject CNAMEs at root.
Attach WAF to ALB; enable AWS Managed Rules (free of charge).	CWASRP (CloudWatch Advanced Security Rule Processor) replacing ALB — vulnerable to SQL injection, XSS, and bot attacks.
Enable ALB access logs to S3; query with Athena for debugging.	Access logs to CloudWatch — when 503s appear, no data to diagnose which endpoints are failing.
Use health check path that verifies actual app readiness (e.g. /health).	Health check always returns 200 — ALB routes traffic to tasks that cannot serve requests.
Enforce HTTPS: redirect HTTP:80 to HTTPS:443 at ALB.	Allowing HTTP:80 traffic — PII and session tokens transmitted in plaintext.
Set deregistrationDelay=30s; ensure requests completed before task deregistered.	deregistrationDelay=0 — in-flight requests cut off when task deregisters; 502 errors.

ALB listener rules can do more than just forward traffic. ShopFlow uses ALB to return fixed responses for maintenance mode, redirect legacy URLs, and add security headers to all responses.

```
// CDK: Advanced ALB listener rule actions // 1. Fixed response for maintenance mode (toggle by
// changing rule priority) httpsListener.addAction("MaintenanceRule",
AddApplicationActionProps.builder() .priority(0) // Priority 0 = highest; evaluated first
.conditions(List.of( ListenerCondition.httpHeader("X-Maintenance", List.of("true"))))
.action(ListenerAction.fixedResponse(503, FixedResponseOptions.builder()
.contentType("text/html") .messageBody("ShopFlow is under maintenance. Back soon!") .build()))
.build()); // 2. Redirect legacy API URLs httpsListener.addAction("LegacyRedirect",
AddApplicationActionProps.builder() .priority(10)
.conditions(List.of(ListenerCondition.pathPatterns(List.of("/api/v1/*"))))
.action(ListenerAction.redirect(RedirectOptions.builder() .host("#{host}")
.path("#{path}".replace("/api/v1/", "")) // strip /api/v1 prefix .query("#{query}")
.statusCode("HTTP_301") .build())) .build()); // 3. Authentication: Cognito OIDC authentication
before forwarding to admin panel httpsListener.addAction("AdminAuth",
AddApplicationActionProps.builder() .priority(5)
.conditions(List.of(ListenerCondition.pathPatterns(List.of("/admin/*"))))
.action(ListenerAction.authenticateCognito( AuthenticateCognitoOptions.builder()
.userPool(adminUserPool) .userPoolClient(adminUserPoolClient)
.userPoolDomain(adminUserPoolDomain)
.onUnauthenticatedRequest(UnauthenticatedAction.AUTHENTICATE) .build(),
ListenerAction.forward(List.of(adminTargetGroup)))) .build());
```

AWS Global Accelerator provides static anycast IP addresses that route traffic over the AWS global network backbone instead of the public internet. For ShopFlow international customers, this reduces latency by 30-60% by entering the AWS network at the nearest edge location.

Aspect	Without Global Accelerator	With Global Accelerator	
Traffic path	Browser -> public internet -> us-east-1 ALB (full internet path)	Browser -> nearest AWS edge (125+ PoPs) -> AWS backbone -> us-east-1 ALB	
IP addresses	ALB DNS name (changes if ALB changes)	2 static anycast IPs (never change)	
Latency (Europe user)	~120ms round-trip to US-EAST-1 via internet	~60-80ms (enter AWS network in Europe; fast backbone to US)	
Failover	Route 53 DNS TTL (60s failover)	< 30 seconds (traffic level, not DNS)	

Aspect	Without Global Accelerator	With Global Accelerator	
Cost	No extra charge	\$0.025/hour (\$18/month) + \$0.01/GB	
ShopFlow use case	Not yet implemented (most users are US-based)	Recommended when expanding to Europe/Asia	

INFO CloudFront vs Global Accelerator

Both improve global performance but differently. CloudFront caches content at edge locations — great for static assets, images, and API responses that can be cached. Global Accelerator does not cache — it only accelerates network routing. Use CloudFront for ShopFlow product images and API cache. Use Global Accelerator for non-cacheable requests like checkout and order history that must always go to the origin.

13-18

ALB and ECS Fargate — Dynamic Port Mapping

One subtlety of ALB + ECS Fargate: Fargate tasks use awsvpc networking (each task gets its own IP), so ALB registers tasks by private IP address — not by instance ID or port. This differs from EC2 where multiple tasks on one instance share an IP with different ports.

Aspect	EC2 Launch Type	Fargate Launch Type (ShopFlow)
Network mode	bridge mode: tasks share EC2 host network address in host pods	awsvpc mode: each task has its own ENI and private IP
Target type	instance (EC2 instance ID)	ip (task private IP address)
Port mapping	Dynamic: ALB -> host:random_port -> container:8080	Direct: ALB -> task_ip:8080 (no port mapping needed)
Security group	On the EC2 instance	On the task ENI directly
Task count limit	Multiple tasks per EC2 instance (limited by host capacity)	One task per private IP; limited by subnet IP space

```
// Key CDK setting for Fargate + ALB: TargetType.IP (not INSTANCE)
ApplicationTargetGroup tg = ApplicationTargetGroup.Builder.create(...)
    .targetType(TargetType.IP) // REQUIRED for Fargate
    .port(8080) .build(); // ECS Service registers tasks directly (no explicit addTargets needed)
FargateService svc = FargateService.Builder.create(...) .build();
svc.attachToApplicationTargetGroup(tg); // ECS automatically registers each task's private
IP:8080 with the target group // when the task starts and deregisters when it stops
```

13-19

Route 53 Private Hosted Zones — Internal DNS

Private hosted zones provide DNS resolution within a VPC. ShopFlow uses private DNS for internal service discovery that is not handled by ECS Service Connect, and to resolve custom names for RDS and ElastiCache endpoints.

```
// CDK: Private hosted zone for internal ShopFlow services
PrivateHostedZone internalZone = PrivateHostedZone.Builder.create(this, "InternalZone")
    .zoneName("shopflow.internal") .vpc(vpc)
    // only resolvable within this VPC
    .build(); // Create CNAME for Aurora cluster endpoint
CnameRecord.Builder.create(this, "AuroraRecord") .zone(internalZone)
    .recordName("db-primary.shopflow.internal")
    .domainName(auroraCluster.getClusterEndpoint().getHostname())
    .ttl(Duration.seconds(30)) // short TTL: failover changes hostname; catch it quickly
    .build(); // Read replica endpoint
CnameRecord.Builder.create(this, "AuroraReadRecord") .zone(internalZone)
    .recordName("db-read.shopflow.internal")
    .domainName(auroraCluster.getClusterReadEndpoint().getHostname())
    .build(); // Application uses: jdbc:postgresql://db-primary.shopflow.internal:5432/shopflow
// If Aurora endpoint hostname changes (failover), only update the CNAME // – no application configuration change needed
```

Private DNS Use Case	Internal Name	Resolves To
Aurora write endpoint	db-primary.shopflow.internal	shopflow-cluster.cluster-xyz.us-east-1.rds.amazonaws.com
Aurora read replicas	db-read.shopflow.internal	shopflow-cluster.cluster-ro-xyz.us-east-1.rds.amazonaws.com
ElastiCache Redis	redis.shopflow.internal	shopflow-redis.xyz.cache.amazonaws.com (config endpoint)
Internal admin service	admin-svc.shopflow.internal	Internal ALB DNS name for admin tools

Route 53 and ALB costs are straightforward. The main variable costs are ALB capacity units (driven by connections and request complexity) and Route 53 query volume.

Component	Pricing	ShopFlow Monthly Volume	Monthly Cost
Route 53 Hosted Zone	\$0.50/hosted zone/month	1 public + 1 private	\$1.00
Route 53 DNS Queries	\$0.40/million (first 1B)	50M queries/month (caching reduces actual queries)	\$20.00
Route 53 Health Checks	\$0.50/health check/month	3 health checks (primary + DR + admin)	\$1.50
ALB Fixed Cost	\$0.008/ALB/hour	1 ALB x 730 hours	\$5.84
ALB Capacity Units (LCU)	\$0.008/LCU/hour	~5 LCUs at normal traffic	\$29.20
ACM Certificate	Free	1 wildcard cert (*.shopflow.com)	\$0
WAF WebACL	\$5/WebACL/month + \$1/rule	1 WebACL + 3 rules	\$8
WAF Requests	\$0.60/million requests	100M requests/month	\$60
TOTAL	—	—	~\$125/month

ShopFlow plans to expand to Europe (eu-west-1) and Asia (ap-southeast-1). Route 53 latency-based routing and failover routing enable the multi-region architecture without changing application code.

■ Configure Multi-Region with Route 53 Latency Routing

1. Deploy ShopFlow ECS stack to eu-west-1 (Ireland) — same CDK, different region
2. Create ALB in eu-west-1 for European traffic
3. Route 53 -> shopflow.com A record -> Routing policy: Latency
4. Add record 1: ALIAS -> us-east-1 ALB, Region: us-east-1
5. Add record 2: ALIAS -> eu-west-1 ALB, Region: eu-west-1
6. Route 53 automatically routes each user to the region with lowest latency
7. US East Coast users -> us-east-1 (~20ms), European users -> eu-west-1 (~15ms)
8. Add health checks to both records: if one region ALB fails, Route 53 routes all to the healthy region
9. Test: use VPN to appear as UK user -> verify DNS returns eu-west-1 ALB IP

Region	Users Served	ALB Endpoint	Aurora Read Replica?
us-east-1 (primary)	US, Canada, Latin America	shopflow-alb.us-east-1.elb.amazonaws.com	Primary cluster here
eu-west-1 (secondary)	Europe, Middle East, Africa	shopflow-alb.eu-west-1.elb.amazonaws.com	Aurora Global DB read replica
ap-southeast-1 (planned)	Asia Pacific	shopflow-alb.ap-southeast-1.elb.amazonaws.com	Aurora Global DB read replica

In this lab you will register a test domain, create a Route 53 hosted zone, issue an ACM certificate, and point it to an existing ALB. Time estimate: 45 minutes.

■ Lab: Route 53 + ACM + ALB Setup

1. Register a domain: Route 53 -> Registered domains -> Register (or use a free .tk domain from Freenom)
2. Create hosted zone: shopflow-lab.com (or your domain)
3. Update registrar NS records to point to Route 53 NS records (takes up to 48h to propagate)
4. Request ACM certificate: shopflow-lab.com + *.shopflow-lab.com (DNS validation)
5. Create Route 53 CNAME records for ACM validation: ACM Console -> Create records in Route 53
6. Wait for certificate status: Issued (~5 minutes)
7. Create ALB: shopflow-lab-alb (internet-facing, HTTPS:443)
8. Attach ACM certificate to HTTPS listener
9. Add listener rule: /products/* -> product-svc target group
10. Create Route 53 A record: shopflow-lab.com -> ALIAS to shopflow-lab-alb
11. Test: <https://shopflow-lab.com/products/search?q=shoes> (should hit ECS)
12. Cleanup: delete ALB, ACM certificate, Route 53 records, hosted zone

13-23 ALB Connection Draining and Slow Start

When ECS deploys a new ShopFlow version, Fargate tasks are replaced one by one. Two ALB features make this seamless: Deregistration Delay (connection draining) and Slow Start Mode.

Feature	What it Does	ShopFlow Setting	Why It Matters
Deregistration Delay (connection draining)	When a target is deregistered, ALB waits up to 300 seconds (default) for requests to complete before removing it.	300 seconds (default)	Prevents "complete request refused" errors.
Slow Start Mode	Newly registered targets receive a linearly increasing share of requests over 60 seconds.	60 seconds	Prevents initial traffic spikes from causing failures.
Health check grace period	ECS waits N seconds after task starts before ALB health checks begin.	20 seconds	Spring Boot takes ~30-45s to start; without this, it would fail.

```
// CDK: Configure deregistration delay + slow start on target group ApplicationTargetGroup tg =
ApplicationTargetGroup.Builder.create(this, "ShopFlowTG") .vpc(vpc) .port(8080)
.protocol(ApplicationProtocol.HTTP) .targetType(TargetType.IP)
.healthCheck(HealthCheck.builder() .path("/actuator/health") .healthyHttpCodes("200")
.interval(Duration.seconds(30)) .healthyThresholdCount(2) .unhealthyThresholdCount(3) .build())
.deregistrationDelay(Duration.seconds(300)) // drain connections before removing task
.slowStart(Duration.seconds(60)) // ramp up new tasks gradually .build(); // ECS service health
check grace period (separate from ALB) FargateService svc = FargateService.Builder.create(this,
"ShopFlowSvc") .cluster(cluster) .taskDefinition(taskDef)
.healthCheckGracePeriod(Duration.seconds(120)) // Spring Boot startup time .build();
```

13-24 ALB Access Logs Analysis — Finding Performance Issues

ALB access logs record every request to S3. ShopFlow's SRE team uses Athena to query these logs and identify slow product searches, checkout errors, and bot traffic patterns.

```
-- Athena: Create table for ALB access logs CREATE EXTERNAL TABLE IF NOT EXISTS alb_logs ( type
string, time string, elb string, client_ip string, client_port int, target_ip string,
target_port int, request_processing_time double, target_processing_time double,
response_processing_time double, elb_status_code int, target_status_code int, received_bytes
bigint, sent_bytes bigint, request string, user_agent string, ssl_cipher string, ssl_protocol
string, target_group_arn string, trace_id string, domain_name string, chosen_cert_arn string,
matched_rule_priority string, request_creation_time string, actions_executed string,
redirect_url string, error_reason string ) ROW FORMAT SERDE
"org.apache.hadoop.hive.serde2.RegexSerDe" WITH SERDEPROPERTIES ("input.regex" = "([^ ]*) ([^
]*) ([^ ]*) ([^ ]*):([0-9]*) ([^ ]*)[:-]([0-9]*) ([-\\.0-9]*) ([-\\.0-9]*) ([-\\.0-9]*) ([|[-0-9]*)
```

```
(-|[-0-9]*) ([-0-9]*) ([-0-9]*) "([^\ ]*) (.*) (-|[-0-9]*)" "([^\"]*)" ([A-Z0-9-_-]+)
([A-Za-z0-9-_-]*) "([^\ ]*)" "([^\"]*)" "([^\"]*)" "([^\"]*)" "([^\"]*)" "([^\"]*)" "([^\"]*)" "([^\"]*)"
"([^\ ]*)" "([^\s]+?)" "([^\s]+)" "([^\ ]*)" "([^\ ]*)"") LOCATION
"s3://shopflow-alb-logs/AWSLogs/123456789/elasticloadbalancing/us-east-1/"; -- Query: Find
slowest ShopFlow product search requests SELECT request, target_processing_time,
elb_status_code, client_ip FROM alb_logs WHERE target_processing_time > 2.0 -- requests taking
over 2 seconds AND request LIKE "%/api/products/search%" -- product search requests only AND
elb_status_code = 200 -- successful requests only ORDER BY target_processing_time DESC LIMIT
20;
```

13-25

Troubleshooting Route 53 and ALB — Common Issues

These are the most common Route 53 and ALB issues ShopFlow engineers encounter, with diagnostic commands and fixes.

Symptom	Likely Cause	How to Diagnose	Fix
DNS change not taking effect	Old DNS still cached by resolver	dig+nocache shopflow.com @8.8.8.8	Wait for TTL to expire (lower TTL before making change)
502 Bad Gateway from ECS	Task crashed; port 8080 not listening	Check target group health checks: EC2ApplicationGroup	Verify Target is healthy
SSL certificate error	Certificate not attached to ALB	Check TLS Listener: port 443 certificate	Attach ACM certificate to ALB listener; check cert domain
Health check always fails	Wrong path or port in health check	ALB Target Group -> Attributes -> HealthCheckPath	Match path to test script; health check security group
Route 53 failover not working	Health check passing (ALB returns 200)	Check health check script; test health check directly	Test health check script; if it should return 503
High 4xx errors in ALB	CDN errors or WAF blocking	Check CloudFront logs; HTTP 4xx; Legitimate Monitoring	Check WAF/Simple Rules; Legitimate User-Agents blocked

13-26

Black Friday Readiness — Traffic and DNS Checklist

Route 53 and ALB are in the critical path for every ShopFlow request. These pre-Black Friday checks ensure traffic management can handle 10x load.

Check	Normal Traffic	Black Friday Requirement	Action Needed
ALB LCU capacity	~5 LCUs	~50 LCUs (10x connections + data)	ALB scales automatically; pre-warm request to AWS Support
Target group healthy targets	6 ECS tasks	60 ECS tasks (ECS Auto Scaling)	Verify Auto Scaling policy triggers at 60% CPU; test scaling
ALB access log latency	~900ms	< 1000ms acceptable (2x budget)	Run Athena query on access logs week before; identify anomalies
Route 53 health check sensitivity	3 failures x 30s = 90s to failover	3 failures x 10s = 30s (update for BFD)	Reduce health check interval from 30s to 10s one week before
WAF rate limiting	1000 req/IP/5min	5000 req/IP/5min (CDN origin IP addresses)	Add CloudFront IP ranges to WAF allowlist; adjust rate limit
Certificate expiry	Check monthly	Verify > 60 days remaining on ACM cert	ACM auto-renews DNS-validated certs; verify auto-renewal
SSL/TLS policy	Current	TLSv1.2 minimum; disable TLS 1.0	ALB listener -> Edit -> Security Policy: ELBSecurityPolicy

SUCCESS Black Friday Simulation Test

Two weeks before Black Friday, run a load test using k6 or Gatling targeting the ALB directly (bypassing Route 53 to eliminate DNS caching effects). Ramp from 100 to 10,000 req/s over 30 minutes. Watch: ALB Target Response Time in CloudWatch, ECS task count (Auto Scaling triggering), Target Group healthy host count, ALB HTTP 5xx rate (should stay < 0.1%). If 5xx spikes appear, check ECS task startup time and health check grace period.

13-27

Well-Architected Review — Traffic Layer

The AWS Well-Architected Framework has specific guidance for DNS and load balancing. ShopFlow's traffic layer is evaluated against the five pillars.

Pillar	Requirement	ShopFlow Status	Reference
Operational Excellence	Use infrastructure as code for all traffic	Only Route 53 records and ALB rules in CDK; S3 deployment via GitHub Actions	

Pillar	Requirement	ShopFlow Status	Reference
Security	Encrypt in transit; authenticate at the edge	■ ACM TLS; WAF WebACL; Cognito authentication	SEC-01 Protect all traffic in transit
Reliability	Health checks with automatic failover; multi-AZ	■ 3 AZs ALB health checks on ALB; ALB spans 3 AZs; EC2 Auto Scaling	RES-03 Multi-AZ architecture
Performance Efficiency	Use caching and CDN for static assets; optimize ALB for static/*	■ CloudFront for static/*; ALB path-based routing	PER-02 Use caching to reduce backend load
Cost Optimization	Right-size ALB; eliminate unused listeners	■ Review: 3 ALB rules are unused from CloudFront Decommission unused resources	COS-01 Decommission unused resources

INFO Next Steps for ShopFlow Traffic Layer

Three improvements are planned for Q1: (1) Enable Global Accelerator for international users once EU expansion is approved. (2) Implement AWS Shield Advanced for DDoS protection above WAF level — ShopFlow processes payments so enhanced DDoS protection is worth the \$3,000/month. (3) Migrate /api/v1/* clients to /api/v2/* and remove the legacy redirect rule from ALB.

13-28

CDK Patterns — Complete Traffic Stack

Here is the complete production-ready CDK construct combining Route 53, ACM, ALB, WAF, and health checks. This is the single source of truth for ShopFlow's traffic layer.

```
// Complete ShopFlow Traffic Stack (CDK construct excerpt) public class ShopFlowTrafficStack extends Stack { public ShopFlowTrafficStack(App app, String id, VpcProps vpcProps) { super(app, id); // 1. Import existing VPC IVpc vpc = Vpc.fromLookup(this, "VPC", VpcLookupOptions.builder().vpcId(vpcProps.vpcId()).build()); // 2. Application Load Balancer (internet-facing, 3 AZs) ApplicationLoadBalancer alb = ApplicationLoadBalancer.Builder.create(this, "ALB").vpc(vpc).internetFacing(true).loadBalancerName("shopflow-prod-alb").build(); // 3. ACM Certificate (DNS-validated; auto-renews) HostedZone zone = HostedZone.fromLookup(this, "Zone", HostedZoneProviderProps.builder().domainName("shopflow.com").build()); Certificate cert = Certificate.Builder.create(this, "Cert").domainName("shopflow.com").subjectAlternativeNames(List.of("*.shopflow.com")).validation(CertificateValidation.fromDns(zone)).build(); // 4. HTTPS listener with ACM certificate ApplicationListener https = alb.addListener("HTTPS", BaseApplicationListenerProps.builder().port(443).certificates(List.of(cert)).build()); alb.addRedirect(ApplicationLoadBalancerRedirectConfig.builder().sourcePort(80).targetPort(443).build()); // 5. WAF WebACL – rate limiting + AWS managed rules CfnWebACL waf = CfnWebACL.Builder.create(this, "WAF").name("shopflow-waf").scope("REGIONAL").defaultAction(CfnWebACL.DefaultActionProperty.builder().allow(Map.of()).build()).visibilityConfig(CfnWebACL.VisibilityConfigProperty.builder().sampledRequestsEnabled(true).cloudWatchMetricsEnabled(true).metricName("ShopFlowWAF").build()).rules(wafRules()).build(); CfnWebACLAssociation.Builder.create(this, "WAFAssoc").resourceArn(alb.getLoadBalancerArn()).webAclArn(waf.getAttrArn()).build(); // 6. Route 53 ALIAS record -> ALB ARecord.Builder.create(this, "ARecord").zone(zone).recordName("shopflow.com").target(RecordTarget.fromAlias(new LoadBalancerTarget(alb))).build(); ARecord.Builder.create(this, "ARecordWWW").zone(zone).recordName("www.shopflow.com").target(RecordTarget.fromAlias(new LoadBalancerTarget(alb))).build(); } }
```

13-29

Chapter Summary — What You Built

In this chapter you configured the complete traffic entry layer for ShopFlow. Every HTTP request from a customer browser passes through the stack you built here.

Topic	Key Takeaway
DNS and Route 53	Route 53 translates shopflow.com to your ALB IP. ALIAS records are free and auto-update when ALB IP
HTTPS with ACM	AWS Certificate Manager issues free TLS certificates with automatic renewal. Attach to ALB HTTPS liste
ALB Listener Rules	Rules are evaluated top-to-bottom by priority. Path-based rules (/api/products/*) route to the right ECS se
Target Groups and Health Checks	Target groups (TargetType.IP for Fargate) hold the ECS task IPs. Health check /actuator/health every 30
Connection Draining	300-second deregistration delay allows in-flight checkout requests to complete during ECS deployments.

Topic	Key Takeaway
WAF Integration	WAF WebACL on the ALB blocks SQL injection, XSS, and rate-limits IPs at > 1000 req/5min. AWS Manag
Route 53 Health Checks and Failover	Route 53 health checks poll the ALB every 10-30s. Failover routing automatically switches to DR region
Private Hosted Zones	shopflow.internal provides stable DNS names for Aurora, Redis, and internal services. Application config
Cost	~\$125/month total for Route 53 + ALB + ACM + WAF at ShopFlow's traffic level. WAF requests (\$60/mo